

SSD: Single Shot Detector

My notes on object detection - architecture and concepts

DEEP LEARNING ROADMAP • OBJECT DETECTION • TOPIC: SSD

SSD is a deep learning model that looks at an image once, and in that single pass, tells us what objects are in it and where exactly they are (a box around each one). That is why it's called a "single shot" detector - no separate step to first guess regions and then classify them, like older methods such as Faster R-CNN do. I'm writing this note the way I understood it, step by step, with the doubts I had along the way and how I resolved them.

What's inside this note

- The full architecture, explained top to bottom
- Why each part exists - not just what it's called
- How multi scale detection actually works (with a visual)
- Default boxes and aspect ratios, explained simply
- How duplicate boxes get cleaned up (NMS) + the two thresholds involved
- Doubts I had while learning this, and how they got cleared up
- A quick comparison with Faster R-CNN
- Practice questions to test myself before moving on

Contents

1. The architecture, top to bottom
2. Step by step - in simple words
3. Default boxes and aspect ratios
4. Non-max suppression and the two thresholds
5. Doubts I had (and how they got cleared)
6. SSD vs Faster R-CNN - quick comparison
7. Summary and practice questions

1. The architecture, top to bottom

Before going into any explanation, here is the full path an image takes through the network - from the raw input, all the way to the final detections.

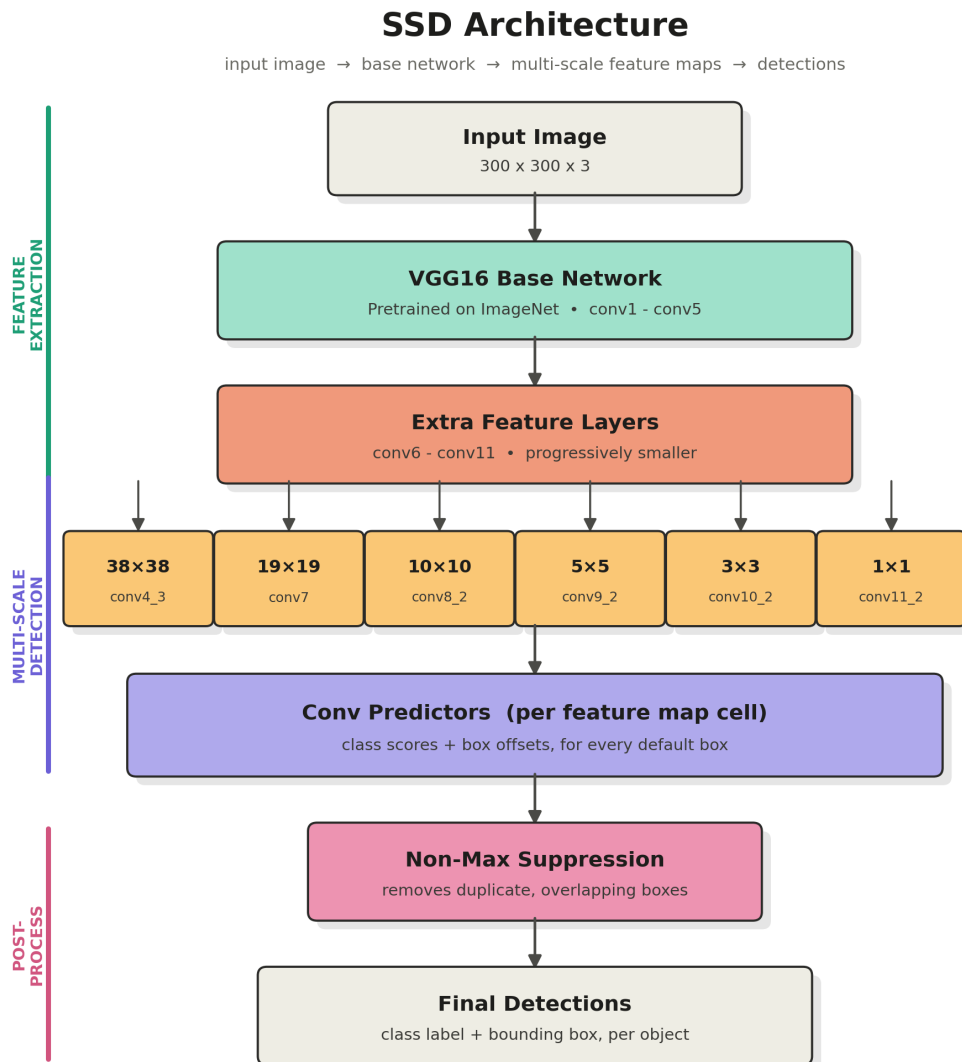


Figure 1: SSD end-to-end pipeline, input to final detections

2. Step by step, in simple words

Here I'm explaining every block of the diagram, one at a time, in the same order the image flows through the network.

Step 1: Input image

- The image is resized to a fixed size, usually 300x300 pixels, before it goes into the network.
- A fixed size is needed because the rest of the network (all the conv layers ahead) expects a consistent input shape.

Step 2: VGG16 base network

- A pretrained VGG16 model is used, but only its convolution layers - the final classifier (fully connected layers + softmax) is removed.
- This part extracts general features like edges, textures and shapes from the image.
- Since it is already trained on a huge dataset (ImageNet), it already knows how to recognise low level visual patterns - we don't need to train this part from scratch.

Why this matters

This is basically transfer learning - reusing a model trained on one big task (classification) as a feature extractor for a different task (detection). It saves a huge amount of training time and data.

Step 3: Extra feature layers (conv6 - conv11)

- More convolution layers are added after VGG16. Each one shrinks the feature map size - 19x19, then 10x10, then 5x5, 3x3, and finally 1x1.
- Each of these layers uses a stride of 2, which is what causes the shrinking (downsampling).
- As the feature map gets smaller, each cell in it 'sees' a larger area of the original image. This growing area is called the receptive field.

Step 4: Multi scale feature maps

- Instead of using only the last (smallest) layer, SSD keeps the output of 6 different layers - it treats each one as a separate 'view' of the image.
- Large feature maps (like 38x38) have a small receptive field, so they are good at finding small objects.
- Small feature maps (like 1x1) have a large receptive field, so they are good at finding large objects.
- This way, objects of every size have at least one layer that is suited to detect them, all in the same forward pass.

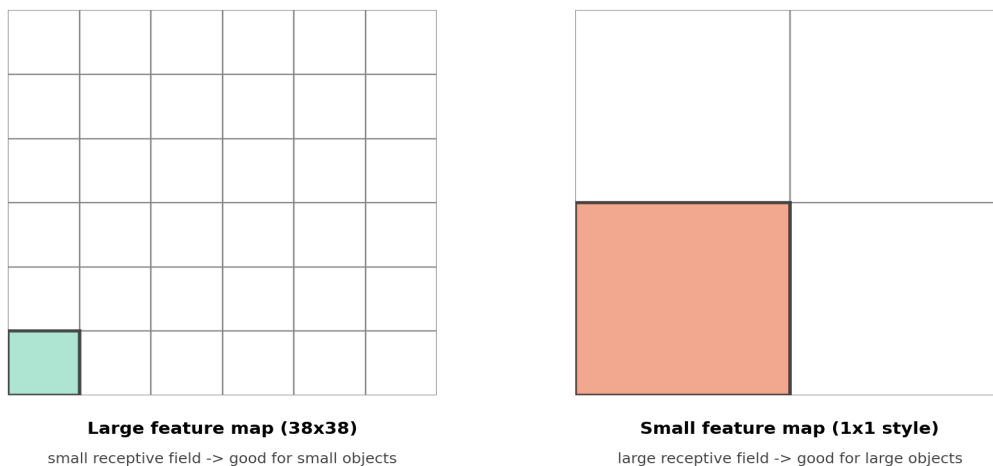


Figure 2: A large feature map = zoomed in view (small objects). A small feature map = zoomed out view (large objects).

Doubt I had: isn't shrinking the feature map just about extracting 'important info'?

Not exactly. The shrinking isn't about compressing information in a generic sense - its specific job is to build a pyramid of different receptive field sizes, so that the network can detect objects at every scale using the layer best suited for that size, instead of forcing one single layer to handle everything.

3. Default boxes and aspect ratios

Step 5: Default boxes (anchor boxes)

- On every single cell of every feature map, a few fixed-shape boxes are placed in advance, before any prediction happens.
- These are called default boxes or anchor boxes - usually 4 to 6 per cell, each with a different shape (aspect ratio): square, wide, tall, and a smaller square.
- They act as starting templates. The network does not draw a box from scratch - it takes a default box and predicts how much to shift and resize it to match a real object.

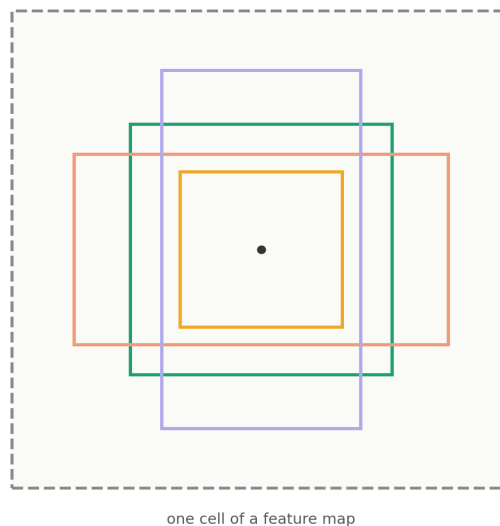


Figure 3: One cell, with 4 default boxes of different aspect ratios placed on it

Step 6: Conv predictors (class scores + box offsets)

- For every default box, two small convolution layers make predictions on top of the feature map.
- The first predicts class scores: what object is inside this box, or is it just background.
- The second predicts box offsets: 4 numbers (shift in x, shift in y, change in width, change in height) that adjust the default box so it fits the real object better.
- This happens for every default box, on every cell, on every feature map - all in one single convolution pass. That is exactly why it's called single shot.

Doubt I had: does the 'info' going into the predictor mean the box itself carries information?

No - the box is just a fixed reference shape/template. What actually goes into the two conv layers is the feature vector at that cell's location (produced by the base network + extra layers). The same feature vector is used to predict both the class score and the box offset for each default box sitting at that cell.

4. Non-max suppression and the two thresholds

Step 7: Non max suppression (NMS)

- Because there are thousands of default boxes (roughly 8700+ for a 300x300 input), many overlapping boxes get predicted for the same real object.
- NMS keeps only the box with the highest confidence score for each object, and removes the rest.
- A box is removed if it overlaps too much (high IoU) with a box that already has a higher confidence score.

Step 8: Final output

- What remains is a clean set of boxes - one per object, each with a class label and a bounding box.

The two thresholds used in NMS

Threshold	What it controls	Typical value
Confidence threshold	Removes boxes the model itself is not confident about	0.5 (or as low as 0.01 for initial filtering)
IoU threshold	Decides if two boxes are duplicates pointing at the same object	0.45

The order matters: first, every box below the confidence threshold is dropped completely. Then, among the boxes that remain, they're sorted by score - and starting from the highest, any box that overlaps more than the IoU threshold with an already-kept box gets removed.

Doubt I had: what's a simple way to picture NMS?

Imagine 5 friends all pointing at the same cat, each with a different confidence: 'I'm 90% sure', 'I'm 70% sure', and so on. NMS says - trust the most confident one, and ignore the others pointing at the same cat. Result: one clean box per object, instead of five overlapping ones.

5. Doubts I had (and how they got cleared)

Collecting all the confusing points from this topic in one place, so I can revisit them quickly later.

Is the base network just 'pretrained model + extract features'?

Yes, that's the core idea - but precisely: the FC classifier layers are removed, only the conv layers stay, and the output is a feature map (like 38x38x512), not a class prediction.

Why shrink the feature map again and again?

Not to compress 'important info' in a vague sense - specifically to increase the receptive field at each stage, so different layers become responsible for different object sizes.

Does every feature map help detect 'every size' of object equally?

No - each feature map is best at one size range. Together, the 6 of them cover the full range from small to large objects.

Does the default box itself carry information into the predictor?

No - the box is only a template shape. The actual input to the predictor is the feature vector at that location; the box offset output tells us how to move/resize that template.

What's the difference between the confidence threshold and IoU threshold?

Confidence threshold filters out boxes the model isn't sure about. IoU threshold filters out duplicate boxes that overlap too much with a better box already kept.

6. SSD vs Faster R-CNN - quick comparison

Since I covered the R-CNN family before this, it helps to place SSD next to Faster R-CNN directly.

	Faster R-CNN	SSD
Stages	Two stage: region proposals, then classify	Single stage: classify + locate in one pass
Speed	Slower	Much faster, real time capable
Small objects	Generally better	Weaker, though multi scale maps help
Accuracy (overall)	Slightly higher, typically	Close, with a small trade-off

In short: Faster R-CNN spends extra time proposing regions first, which usually helps accuracy, especially on small objects. SSD skips that step entirely and predicts densely across the whole image in one go, trading a bit of accuracy for a big speed gain.

7. Summary and practice questions

Key points to remember

- SSD detects objects in a single forward pass - no separate region proposal step.
- A pretrained VGG16 is used as the base feature extractor (transfer learning).
- Extra conv layers shrink the feature map, which grows the receptive field at each stage.
- Predictions are made on 6 different feature map sizes, so both small and large objects get covered.
- Default boxes act as templates on every cell; the network predicts a class score and a position/size offset for each one.
- Non max suppression removes overlapping duplicate boxes, keeping only the best one per object.
- Two thresholds matter in NMS: confidence threshold (filters weak boxes) and IoU threshold (filters duplicates).

Common pitfalls

- Thinking the default box carries information - it doesn't, the feature vector at that location does.
- Assuming the last (smallest) feature map alone is enough - small objects need the larger, earlier feature maps too.
- Mixing up the two NMS thresholds - confidence threshold is about the model's certainty, IoU threshold is about box overlap.

Practice questions

- 1 Hand calculation: for a 5x5 feature map with 6 default boxes per cell and 21 classes, how many total predictions (boxes) does that one feature map produce?
- 2 Coding: implement a function that generates the default boxes (anchor boxes) for a given feature map size and a list of aspect ratios.
- 3 Concept check: explain in your own words why SSD uses 6 feature maps instead of just the last, smallest one.
- 4 Concept check: what would happen to detection of small objects if we removed the 38x38 feature map from the pipeline?